

Conclusão do Marco 1.2: O ambiente virtual do projeto foi estabelecido com sucesso absoluto sob a diretriz da **ADR 005**. O CPython 3.12.14 foi provisionado hermeticamente, o diretório `.venv` foi instanciado em modo seedless (ultraleve, 121 KB), todos os smoke tests da PEP 405 foram aprovados e a salvaguarda de isolamento contra vazamento no Git foi ativada com a criação do `.gitignore` corporativo.

1. Matriz de Componentes e Evidências Técnicas (Passo 1.2)

Componente / Arquivo	Configuração Real Detectada	Status	Impacto Operacional
<code>.python-version</code>	<code>3.12</code> (5 bytes, <code>b'3.12\n'</code>)	Ativo	SSOT do projeto: força o uv, IDEs e GitHub Actions a utilizarem estritamente o Python 3.12.
CPython Hermético	CPython 3.12.14 (AMD64 / OpenSSL 3.5.8)	Provisionado	Alocado em <code>%APPDATA%\uv\python\</code> ; zero impacto no Python 3.14 do Windows.
Ambiente Virtual	<code>estudo_arquitetura_sistemas_pycoder\.venv</code>	Criado	Instanciado em milissegundos via <code>uv venv</code> com pegada de apenas 121 KB.
Isolamento (PEP 405)	<code>include-system-site-packages = false</code>	Hermético	<code>sys.prefix != sys.base_prefix</code> (True); zero contaminação do sistema global.
Scripts de Ativação	<code>activate.ps1</code> , <code>activate.bat</code> , <code>activate</code>	Testados	Ativação validada no PowerShell (variável <code>VIRTUAL_ENV</code> e PATH shadowing funcionais).
Automação <code>uv run</code>	Invocação direta sem ativação prévia	Operacional	O <code>uv run</code> detecta e utiliza o <code>.venv</code> automaticamente, eliminando erro humano.
Salvaguarda <code>.gitignore</code>	<code>.gitignore</code> corporativo criado	Blindado	Bloqueia commit acidental de <code>.venv/</code> , caches do Python/Ruff e arquivos de segredos <code>.env</code> .

2. Checklist das 5 Micro-Fases Concluídas (Passo 1.2)

Micro-Fase 1.2.1: Declaração Formal de Versão (<code>.python-version</code>)	100% CONCLUÍDO
✓ Arquivo <code>.python-version</code> criado com o conteúdo <code>3.12</code> . ✓ Guardrail ativado: o <code>uv</code> passou a recusar formalmente o Python 3.14 do host.	
Micro-Fase 1.2.2: Provisionamento Hermético do CPython 3.12 via uv	100% CONCLUÍDO
✓ Executado <code>uv python install 3.12</code> ; baixado CPython 3.12.14 64-bit oficial (21 MiB). ✓ Binário validado com OpenSSL 3.5.8 nativo e isolamento em <code>%APPDATA%\uv\python\</code> .	
Micro-Fase 1.2.3: Instanciação do Ambiente Virtual (<code>.venv</code>)	100% CONCLUÍDO
✓ Executado <code>uv venv .venv</code> ; ambiente gerado em fração de segundo. ✓ Launcher stubs leves (45 KB) criados; <code>pyvenv.cfg</code> configurado com isolamento total.	

Micro-Fase 1.2.4: Smoke Test do Interpretador (PEP 405)

100% CONCLUÍDO

- ✓ Invocação direta de `.venv\Scripts\python.exe` confirmou Python 3.12.14 ativo.
- ✓ Teste de ativação com `Activate.ps1` validou injeção e remoção limpa de `VIRTUAL_ENV`.
- ✓ Comando `uv run python` executou perfeitamente mirando o ambiente virtual local.

Micro-Fase 1.2.5: Salvaguarda de Isolamento & Consolidação

100% CONCLUÍDO

- ✓ Criado o `.gitignore` corporativo protegendo `.venv/`, caches e arquivos `.env`.
- ✓ Histórico registrado na `MEMORIA.md` e Tasklist mestre atualizado.

3. Próximo Passo do Roteiro



Transição para o Passo 1.3:

Instalação e configuração do linter/formatter ultrarrápido (`ruff`) e do test runner (`pytest`) dentro do nosso novo ambiente virtual, garantindo o ciclo de feedback automático do Harness.